Cryptmarch Report

WM908-15 Programming and Fundamental Algorithms Report

WM908 Gauntlet Game Report
WM908-15 Programming and Fundamental Algorithms

https://github.com/im-e/Cryptmarch

University of Warwick – WMG

Contents

1 Introduction **2**

2 Controllable Hero, Camera & Menus **2**

2.1 Game loop & delta time . . . 2
2.2 Game state machine . . . 2
2.3 Camera . . . 3
2.4 Playable heroes . . . 4
2.5 FPS counter . . . 4

3 Enemy Entities & Combat **4**

3.1 Entity hierarchy & EntityManager . . . 5
3.2 Dynamic array – Vector<T> . . . 5
3.3 Enemy factory . . . 6
3.4 Spawners . . . 7
3.5 Chase behaviour . . . 8
3.6 Attacks and skills . . . 9
3.7 Top-N targeting spell . . . 10

4 Collision & Grid Systems **11**

4.1 AABB collision . . . 11
4.2 Entity-vs-entity checks . . . 12
4.3 Tile collision & the flat tile array . . . 14
4.4 Data-driven level loading . . . 15
4.5 Fixed vs infinite world . . . 16

5 Collectables, Traps & Advanced Features **16**

5.1 Potions, keys, doors and pressure plates . . . 16
5.2 Traps . . . 18
5.3 Level progression . . . 19
5.4 Multiplayer . . . 19
5.5 Saving and loading . . . 20

6 Evaluation **20**

7 Limitations **21**

8 What I Would Do Differently **21**

9 Conclusion **21**

1 Introduction

Cryptmarch is a game inspired by *Gauntlet* [1]. One or two players pick from three heroes and fight through dungeons, using their hero's abilities to defeat enemies, destroy spawners, manoeuvre through traps and collect items on the way to the exit. There is also an infinite mode, where players try to survive as long as they can and kill as many enemies as possible.

Cryptmarch was written in C++ in Visual Studio Community using the GamesEngineeringBase framework, without any STL containers. Hero, tile and enemy art comes from the 32rogues pack by Seth [2], and the UI uses Kenney's Desert Shooter Pack [3].

2 Controllable Hero, Camera & Menus

2.1 Game loop & delta time

The game follows an update–draw cycle. In the play state, every entity, object and timer is updated and then drawn. All updates use a capped delta time.

```
float deltaTime = timer.dt(); //get time since last frame
float trueDelta = deltaTime; //get the true delta time for fps measurement
//cap delta time to prevent unintended behaviour during freezes
if (deltaTime > 0.05f) //check for very long frames
{
   deltaTime = 0.05f; //cap the delta time to be shorter
} //trade-off being that the game might run in slow mo during low frames
```

The cap protects against lag spikes, for example when a new map loads or the PC comes under sudden heavy load. Lag can cause a large delta time, which moves objects a large distance in a single update. This can cause tunnelling, where an object passes straight through a wall. The trade-off for this is that the game can briefly run in slow motion during a spike, but I would rather the game run slowly and correctly than at full speed and incorrectly.

2.2 Game state machine

Cryptmarch uses a finite state machine (FSM) to move between menus and gameplay. An FSM is always in exactly one of a finite set of states, and transitions to another state when needed. Each stage of the game is a value of the `GameState` enum, which the `GameController` owns. The intro is the MenuState, and the loop goes through menu->selection->play->tally->menu.

```
enum GameState
{
    MenuState, PlayerSelectState, CharacterSelectState, P2CharacterSelectState,
    ModeSelectState, PlayState, TallyState
};
```

The main program loop calls a switch on the current state and only runs that state's update and draw calls. Changing state is a single setter call, so transitions are $O(1)$.

```
switch (gameController.getState())
{
case MenuState: {}
case PlayerSelectState: {}
// ...
case PlayState: {}
case TallyState: {}
}
gameController.setState(PlayerSelectState); //go to player select state
```

An alternative I considered was a set of Boolean flags checked with if/else statements. I started this way in `main`, but it became messy as more states were added, and it was easy to end up with

two flags set at once. An FSM rules that out because only one state can be active.

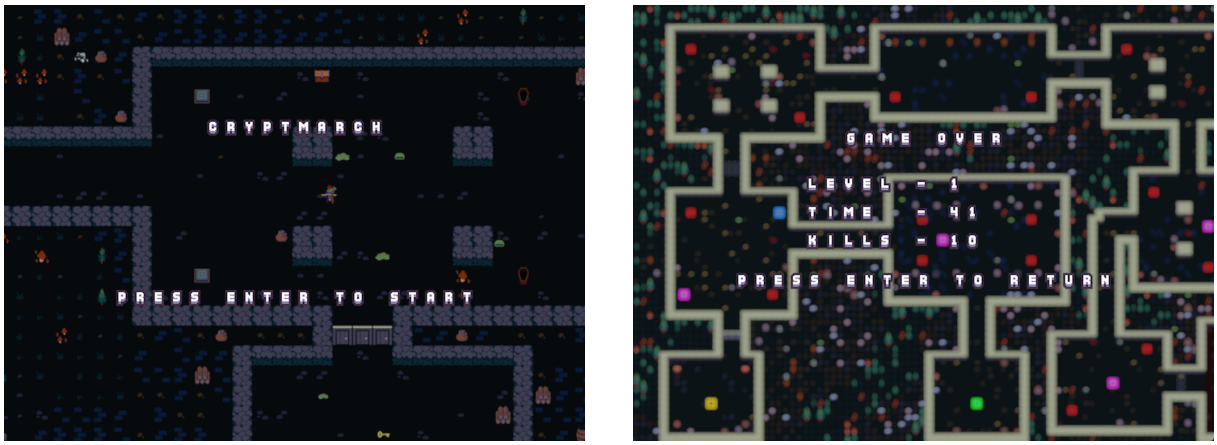


Figure 1: The main menu and the tally screen, two states of the FSM

2.3 Camera

The camera uses a world-to-screen transformation. It has its own world position, and subtracting that from an object's world position gives where the object should be drawn in the window.

```
int worldToScreenX(int worldX) const //convert world x to screen x
{
    //by subtracting the cameras position from the world position we find out where the
    object should be drawn on the window
    return worldX - camX;
}

int worldToScreenY(int worldY) const //convert world y to screen y
{
    return worldY - camY;
}
```

Each frame, the camera's `follow()` function centres it on a target. In multiplayer the target is the midpoint between the two players, and if one player dies the camera locks onto the survivor.

```
if (p1 != nullptr && p2 != nullptr) //if both aren't null
{
    float midpointX = (p1->getX() + p2->getX()) / 2; //get the midpoint of both players
    float midpointY = (p1->getY() + p2->getY()) / 2;
    cam.follow(window, map, midpointX, midpointY, !map.isInfinite());
}
else if (p1 != nullptr) //otherwise prioritize p1 camera
    cam.follow(window, map, p1->getX(), p1->getY(), !map.isInfinite());
else if (p2 != nullptr) //otherwise p1 is dead, so follow p2 instead
    cam.follow(window, map, p2->getX(), p2->getY(), !map.isInfinite());
```

In story mode the camera is clamped to the map boundary, and in infinite mode it isn't. The map's `isInfinite()` flag controls this. The renderer only draws tiles the camera can see, so drawing is $O(\text{visible tiles})$ rather than $O(\text{map size})$.

I considered a flip-screen camera, which jumps a whole screen when the player walks off the edge. However, this breaks in multiplayer as soon as one player runs ahead into the next room. So this was my best choice.

2.4 Playable heroes

There are three heroes: Archer, Dwarf, and the Wizard. I went with the Entity driven approach, where every object in the game with behaviour derives from `Entity` (Section 3.1). `Player` derives from `Entity`, and each hero derives from `Player`, overriding `update`, `draw` and `processAttack`.

Each hero has its own sprite, primary attack, secondary skill, health and mana values, but they all share the same base behaviour which is a controllable character that attacks, updates and draws. One alternative was a data table, which is what I used for enemies. The issue with it is that my heroes differ in behaviour, not just numbers. So a single class would be full of branches that skip code meant for the other heroes. An ECS would be more cache-friendly at thousands of entities, but is overkill for three heroes and hard to build without STL.

2.5 FPS counter

The FPS counter is calculated from delta time and drawn with a bitmap font to the screen. `calculate(dt)` counts frames and adds up passed time, and only recalculates the value every 0.1 seconds. This is so that the number doesn't change hundreds of times per second and is actually readable for the player.

```
double calculate(float dt)
{
    frames++; //increase frame amount
    timeElapsed += dt; //add the time elapsed
    if (timeElapsed > 0.1) //if 0.1s has passed
    {
        fps = frames / timeElapsed; //divide the frames by the time elapsed
        frames = 0; //reset for next calculation
        timeElapsed = 0; //reset time elapsed
    }
    return fps; //return fps
}
```

The FPS class holds a 2D array of tile coordinates for the digits 0–9 on the HUD tilesheet.

```
int fpsNumTileOffsets[2][10] = {
    { 3,4,5,6,7,8,9,10,11,12 }, //x positions
    { 5,5,5,5,5,5,5,5,5,5 }     //y positions
};
```

The label is drawn, and then FPS is then drawn right to left. I take the final digit of the number by using the % operator on the fps value, and draw each digit one by one.

```
while (value > 0)
{
    int digit = value % 10; //get the final digit of the number
    value /= 10; // divide value by 10
    Graphics::DrawTilemapImage(window, *image, tileSize, fpsNumTileOffsets[0][digit],
    fpsNumTileOffsets[1][digit], finalDigitXPos, fpsYPos); //draw digit on screen
    finalDigitXPos -= tileSize; // step left for the next digit
}
```

This was one of the first drawing features I wrote, so it is quite simple. Later I wrote a general text-drawing function that can draw any character on the tilesheet, but I never moved the FPS class over to it.

3 Enemy Entities & Combat

3.1 Entity hierarchy & EntityManager

Every object with behaviour derives from `Entity`. The theory behind it is that all entities have a world position, health, a hitbox, and update and draw functions. This means that shared behaviour between entities, such as taking damage and checking collisions, is written once instead of in every subclass.

The `EntityManager` owns every entity on the heap. It creates and destroys them, calls their update and draw functions, and stores each type in its own `Vector` so that each type can be handled according to its needs. Spawners don't create enemies themselves. They flag that they want to spawn, and the manager does it, so nothing else owns entity memory.

```
class EntityManager {

private:
    Vector<Player*> players; //list of players
    Vector<Enemy*> enemies; //list of enemies
    Vector<Spawner*> spawners; //spawners of enemies
    Vector<Projectile> projectiles; //projectiles in the level
    Vector<Pickup*> pickups; //potions dropped by enemies, and the level key
    Vector<Door*> doors; //exit door and pressure plate doors
    Vector<PressurePlate*> plates; //real and fake pressure plates
    Vector<SpikeTrap*> traps; //spike traps
}
```

The downside of this approach is that some entities carry data they don't need. Traps, for example, have health they never use. An alternative to this would be to have one large entity struct with a type tag and a switch. But this would be far worse as every projectile would carry every field of every entity type, wasting memory.

Instead of separate lists in `EntityManager`, I could have kept a single `Vector<Entity*>`. That makes updating and drawing simpler, but would make collision worse. Finding projectile to enemy hits would mean checking every pair, which is $O(n^2)$ for n entities, plus a type check on each. Separate lists mean I only check the pairs that matter, which is $O(p \cdot e)$ for p projectiles and e enemies.

3.2 Dynamic array – Vector<T>

Since STL containers were banned, I wrote my own dynamic array, `Vector<T>`. `add` takes constant $O(1)$ time on average, as memory allocation and copying costs are spread out. `remove` keeps the order and is $O(n)$, while `removeSwap` moves the last element into the gap and is $O(1)$. I use `removeSwap` when deleting dead entities, looping backwards so the swapped-in element is never skipped.

```
Vector() //constructor
{
    size = 0; //set size to 0
    capacity = 8; //set capacity to small fixed number
    //set to 8 since its a power of 2 and covers general small containers while not being
    too large
    data = new T[capacity]; //allocate a block of that capacity
}
```

The starting capacity is 8, which covers most small lists without wasting memory, and the capacity doubles when the array is full.

```
void add(const T& value) //add item
{
    //if size equals or is larger than capacity
    if (size >= capacity)
    {
```

```
        //change capacity
        changeCapacity(capacity * 2); //by a growth factor of two
        //this means our data will have to grow less than 1.5x. 1.5x could save more memory but
        //using 2x means that expensive grows get further and further apart
    }
    data[size] = value; //store value into slot
    size++; //increase in size
}
```

I chose 2× over 1.5× because expensive copies would get further apart as the array grows. I was expecting lots of enemies spawning, and I didn't want frequent growth which could cost live performance, even though 1.5× uses less memory. Copying is disabled (the rule of three), because two `Vectors` sharing one buffer would both try to delete it.

3.3 Enemy factory

Cryptmarch has six enemy types. They differ in strength but behave mostly the same, so I used a data-driven design: every enemy is the same `Enemy` class, configured from a table of `EnemyDefinitions`.

```
//health, speed, damage, {sprite x, y}, knockback, isRanged, weight, unlock time (seconds)
EnemyDefinition(150, 125, 15, { 0, 2 }, 350, false, 20, 0), //slime - healthy slow low dmg
EnemyDefinition(100, 200, 15, { 4, 6 }, 400, false, 20, 0), //ant - healthy, fast, low dmg
EnemyDefinition(50, 250, 20, { 0, 5 }, 250, false, 18, 20), //ghost - very weak but fast and med dmg
EnemyDefinition(75, 130, 25, { 1, 4 }, 400, true, 16, 40), //skeleton - weak but fast ranged
EnemyDefinition(250, 80, 45, { 0, 1 }, 700, false, 14, 60), //ogre - high health high dmg but slow
EnemyDefinition(300, 50, 35, { 9, 7 }, 550, true, 12, 90), //medusa - high health, slow, high dmg ranged
```

The `EnemyFactory` creates every enemy from this table. Each type has a spawn weight (totalling up to 100) and an unlock time. This makes the game have a scaling difficulty, as easy enemies only appear early and spawn more frequently in general, and the harder enemies appear less frequently and only appear 60–90 seconds in, as levels are designed to be around 2 minutes long. Game time carries over between levels, so the second level starts harder too.

```
EnemyType weightedRandomEnemyType(float* gameTime) //returns a weighted randomised enemy
{
    int unlockedWeight = 0; //weight we will use to divide weight by
    for (EnemyDefinition& enemy : enemyTypes) //for each enemy in enemy types
    {
        if (*gameTime >= enemy.timeUnlocked) unlockedWeight += enemy.weight; //unlocked, add its weight
        else break; //not unlocked yet
    }
    int weight = rand() % unlockedWeight; //get a random number based on enemies unlocked
    int weightSum = 0; //sum of weights
    int randomEnemy = 0; //the random enemy we will pick at the end
    for (int i = 0; i < 6; i++) //for each enemy type
    {
        weightSum += enemyTypes[i].weight; //add up the sum of their weights
        if (weightSum > weight) { randomEnemy = i; break; } //spawn that enemy
    }
    return static_cast<EnemyType>(randomEnemy);
}
```

I could have given each enemy its own subclass, like I did with the players, but they only differ in numbers and whether they are ranged. That approach would have resulted in six almost identical

classes. I also could have had each spawner call `new Enemy(...)` directly, but then enemy stats would be copied around the code, and changing one would mean hunting down every copy.

3.4 Spawners

Spawners are damageable, static enemy–spawning entities.



Figure 2: A spawner (coffin), which recently spawned an Ant Enemy

They only spawn while a player is in range. Spawners use a timer that tracks the time since the last spawn, and they stop spawning once damaged enough to be destroyed.

```
class Spawner : public Entity
{
private:

    float spawnTimer = 0; //timer for spawns
    float spawnInterval = 7.5f; //interval of enemies spawning

    int spawnerRange = 600; //range of if spawner should spawn

    bool readyToSpawn = false; //if the spawner is ready to spawn

public:

    bool getReadyToSpawn() const { return readyToSpawn; }
    void setReadyToSpawn(bool _readyToSpawn) { readyToSpawn = _readyToSpawn; }

    int getSpawnerRange() const { return spawnerRange; }

    Spawner(EntityStats _stats, GamesEngineeringBase::Image* _image) : Entity(_stats,
    _image) {}

    void draw(GamesEngineeringBase::Window& window, Camera& cam) override;
    void update(GamesEngineeringBase::Window& window, Map& map, float deltaTime) override;
};
```

In multiplayer, a spawner activates if any player is in range, and it updates only once per frame however many players are near.

```
//update all spawners (that have a player in range)
for (int i = 0; i < spawners.getSize(); i++) //update spawners that have a player in range
{
```

```
    //work out range of spawner
    Vector2f sp = spawners[i]->getCentrePoint(); //spawner
    float rangeSquared = spawners[i]->getSpawnerRange() * spawners[i]->getSpawnerRange();
    //get spawner range
    bool anyPlayerInRange = false;

    for (int j = 0; j < players.getSize(); j++)
    {
        Vector2f pl = players[j]->getCentrePoint(); //player

        float dx = sp.x - pl.x; //find the difference of the two points
        float dy = sp.y - pl.y; //player to spawner
        float distanceSquared = (dx * dx) + (dy * dy); //If dx^2 + dy^2 is less than range
    ^2, the player is in range
        if (distanceSquared < rangeSquared) //check if in range
        {
            anyPlayerInRange = true; //set player is in range to true;
            break;
        }

    }

    if (!anyPlayerInRange) //spawner ISNT in range to either player
    {
        continue; //skip to next spawner
    }

    //spawner is in range to a player
    spawners[i]->update(window, map, deltaTime); //update spawners
    if (spawners[i]->getReadyToSpawn()) //if they are ready to spawn
    {
        spawnEnemy({ spawners[i]->getX(), spawners[i]->getY() }); //spawn enemy
        spawners[i]->setReadyToSpawn(false); //set ready to spawn to false
    }

}
```

3.5 Chase behaviour

Enemies have two states: wander and chase. While wandering, an enemy picks a random direction every 1–3 seconds and moves at reduced speed to show its wander state, and will pick a new direction if it hits a wall or the timer expires. When a player comes within its aggro range, it chases them based on the normalised direction vector. It only stops chasing once the target leaves a leash (de-aggro) range. The two ranges are different to prevent enemies from switching between chasing and wandering when a player is on the edge of aggro range.

Each frame, the `EntityManager` picks every enemy's target.

```
Entity* current = enemy->getTarget(); //enemy current target
bool keepCurrent = false; //if we are keeping the current player target or not
Player* best = nullptr; //best is going to the closest player in range
float currentDistSq = FLT_MAX; //current distance squared
float bestDistSq = enemy->getAggroRange() * enemy->getAggroRange(); //sq distance for aggro
     range
float leashSq = enemy->getLeashRange() * enemy->getLeashRange(); //sq distance for leash
    range (de-aggro range)
for (int j = 0; j < players.getSize(); j++) //for each player
{
    Player* player = players[j];
    float dx = player->getCentrePoint().x - enemy->getCentrePoint().x;
    float dy = player->getCentrePoint().y - enemy->getCentrePoint().y;
    float distSq = (dx * dx) + (dy * dy); //squared distance
```

```
13      if (player == current && distSq < leashSq) { keepCurrent = true; currentDistSq = distSq
        ; }
14      if (distSq < bestDistSq) { best = player; bestDistSq = distSq; } //closest player in
        aggro range
15  }
16  if (keepCurrent && best != nullptr && best != current && bestDistSq < currentDistSq)
17      keepCurrent = false; //another player is closer, switch to them
18  if (keepCurrent) enemy->setTarget(current); //keep target as current
19  else { enemy->setTarget(best); } //set target to best
```

An enemy keeps its current target while that player is within leash range, unless another player in aggro range is closer. This also fixed a dangling-pointer bug, where enemies originally stored player 1 permanently and kept pointing at freed memory after that player died. The cost is $O(e \cdot p)$, and with $p \leq 2$ this is effectively $O(e)$.

A better alternative would be pathfinding, such as breadth-first search or A*, so that enemies walk around walls instead of straight at the player. This is expensive for hundreds of enemies without optimisation, and it was a lot of work for a project this size. So I went with the simpler approach.

3.6 Attacks and skills

Each hero has a primary attack and a secondary skill that costs mana.

The Dwarf's primary is a melee attack: a generous hitbox in front of the Dwarf damages and knocks back anything inside it. Knockback takes the vector from the Dwarf to the enemy, normalises it and multiplies it by the knockback strength being applied to the enemy over a short time. The swing direction is locked when the attack starts, so it doesn't flip if the player turns mid-swing. The Dwarf also keeps a list of what it has already hit so nothing is hit twice in one swing. Its secondary is a defence buff that boosts speed and reduces damage taken for a short time. The sprite's hue is blue while the buff is active.

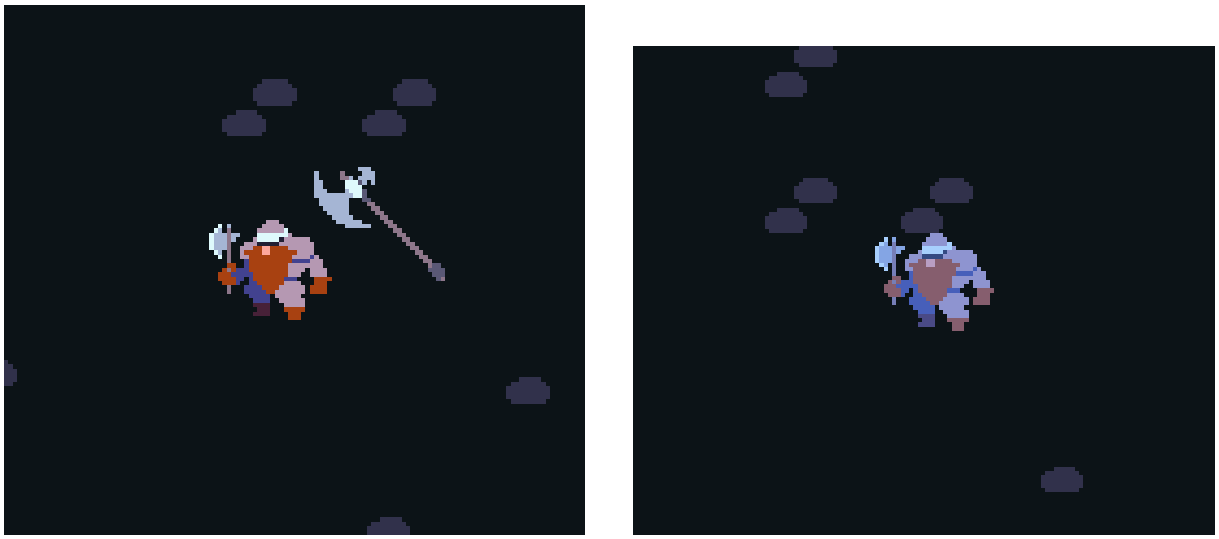


Figure 3: Dwarf primary attack (left) and secondary skill (right)

The Archer's primary fires arrows in the facing direction. The projectiles have a speed, lifetime and damage. `EntityManager` owns the projectiles, along with all the other projectiles in the game, and friendly projectiles only hurt enemies. The Archer's secondary is a dash that moves the player quickly in the facing direction and makes them invulnerable for the duration, again shown with a tint.

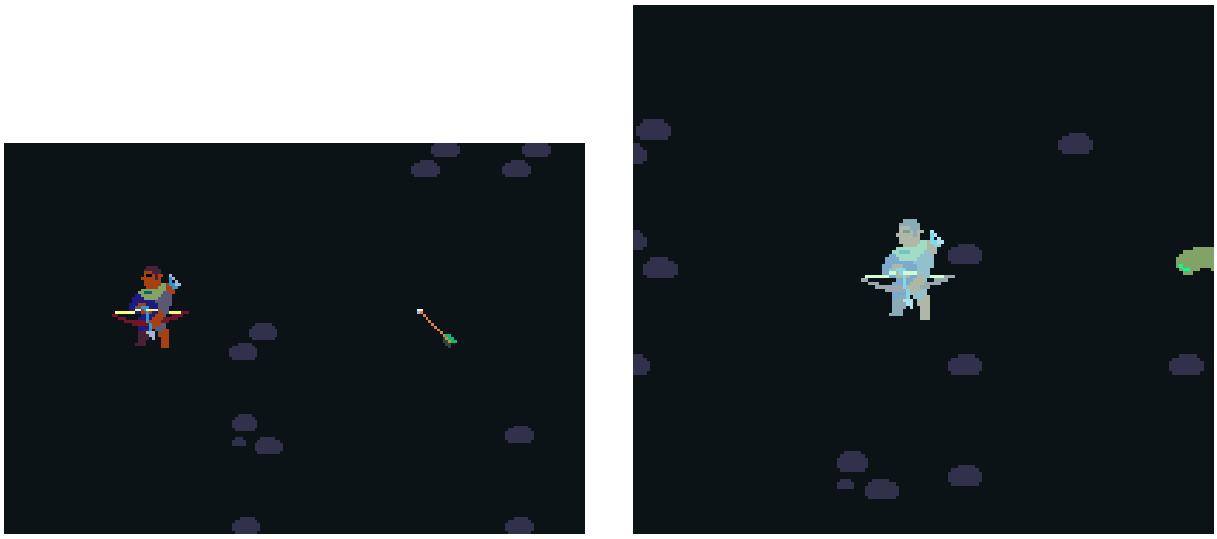


Figure 4: Archer primary attack (left) and secondary skill (right)

The Wizard's primary is a magic nova: an expanding ring that damages enemies around the Wizard and knocks them outwards from its centre.

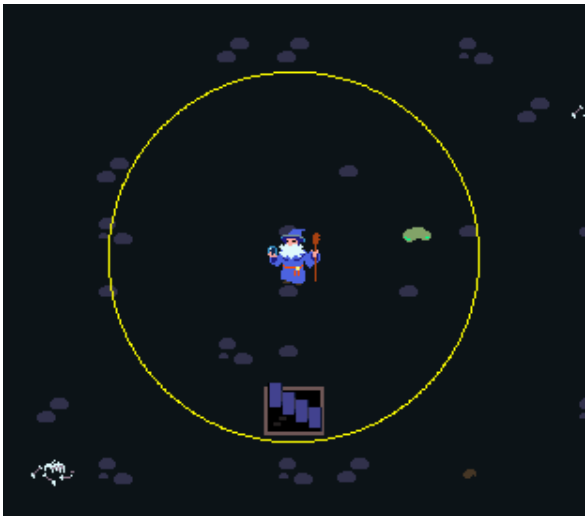


Figure 5: Wizard primary attack

3.7 Top-N targeting spell

The wizards secondary is a Top-N targeting spell, which targets the enemies within a ring who have the highest health. The algorithm uses a bounded insertion leaderboard.

```
1  static int findHighestHPEnemies(Vector<Enemy*>& enemies, Vector2f centre, float range,
       Enemy* board[], int boardSize)
2  {
3      int count = 0; //how many slots in the leaderboard are filled so far
4      for (int i = 0; i < enemies.getSize(); i++)
5      {
6          Enemy* enemy = enemies[i]; //get current enemy
7          int position = 0; //which leaderboard slot the new enemy is "standing in" while it
       shuffles upward
8          if (enemy->isDead()) continue; //skip dead enemies
9          if (!isInRange(enemy, centre, range)) continue; //too far away
10
11         if (count < boardSize) //board isnt full yet
```

```
12          {
13              position = count;
14              count++;
15          }
16          else //board is full: compare against the weakest enemy on the board (the last slot
        )
17          {
18              if (enemy->getHealth() <= board[boardSize - 1]->getHealth()) continue; //weaker
        , skip
19              position = boardSize - 1; //stronger, so start in the last slot (overwrites the
         weakest)
20          }
21          //slide weaker enemies down to make room
22          while (position > 0 && board[position - 1]->getHealth() < enemy->getHealth())
23          {
24              board[position] = board[position - 1]; //move weaker one down one slot
25              position--; //step up one slot towards the top
26          }
27          board[position] = enemy; //place the current enemy where it stopped
28      }
29      return count;
30  }
```

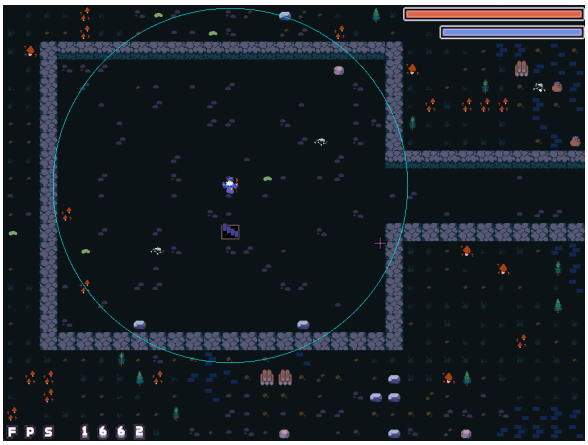


Figure 6: Wizard secondary spell

In terms of complexity, the filter and rejection checks are $O(1)$ per enemy, and the upward shift takes at most N steps, so the worst case (enemies arriving in increasing order of health) is $O(n \cdot N)$ and the best case is $O(n)$. Using <= for rejection and < for the shift means that when two enemies have equal health, the first one keeps its place, which also saves computations for a similar outcome.

An alternative to this could have been to do a min-heap of size N, a min-heap (a binary tree) with the weakest enemy at the root, checked in $O(1)$. New enemies are compared with the root and if its stronger it replaces the root, fixing up the heap in $O(\log N)$. This would have been a better solution if my N was bigger, for example if it was 50 instead of 3, because the upshift process costs $\log N$ instead of N. But because my N is so small (only 3) it meant that this was almost meaningless of a performance increase.

4 Collision & Grid Systems

4.1 AABB collision

Cryptmarch uses axis-aligned bounding box (AABB) collision. Two boxes overlap only if they overlap on both the x and y axes, so a single test is $O(1)$.

```
bool xOverlap = (a.x < b.x + b.width) && (a.x + a.width > b.x);
bool yOverlap = (a.y < b.y + b.height) && (a.y + a.height > b.y);
return xOverlap && yOverlap;
```

Hitboxes aren't stored, they are calculated from the current position of the entity when needed. I went with this approach because firstly, storing them uses up memory, and second- i'd have to update it along with all the movement code, and I didn't want them falling out of sync. The downside is that the hitbox is recalculated every time it is needed, but this is $O(1)$ and doesn't matter for performance.

Movement uses test-then-move, to prevent the position to enter an illegal state for even a short time. The new position is tested on each axis separately and only applied if it isn't blocked. Testing the axes separately lets entities slide along a wall instead of stopping dead.

```
//test if the player is blocked then move
//i could have done move-then-undo but i didnt want my position to enter an illegal state
float newX = worldX + xMove;
if (!map.isBlockedByWall(hitbox.x + xMove, hitbox.y, hitbox.width, hitbox.height) //if not blocked by wall
    && leashAllowsMove(worldX, newX, leashMinX, leashMaxX)) //and leash allows move
{
    worldX = newX; //allow move on x axis
}

hitbox = getHitbox(); //update hitbox after move

float newY = worldY + yMove;
if (!map.isBlockedByWall(hitbox.x, hitbox.y + yMove, hitbox.width, hitbox.height)
    && leashAllowsMove(worldY, newY, leashMinY, leashMaxY))
{
    worldY = newY;
}
```

Enemies use the same isBlockedByWall test-then-move approach (Section 3.5). where hitting a wall makes a wandering enemy pick a new direction.

4.2 Entity-vs-entity checks

Entity collision is a brute force method. Every player is checked against every enemy, and every projectile is checked against every enemy, which is $O(p \cdot e)$. When enemies collide with players, players will take damage and knockback. All entities have a short iframe window, so they can't be damaged again immediately. This prevents a player from taking 1000 damage in one frame from overlapping enemies.

```
//check for collisions between players and enemies
for (int i = 0; i < players.getSize(); i++)
{
    for (int j = 0; j < enemies.getSize(); j++)
    {
        if (players[i]->checkEntityCollision(enemies[j]))
        {
            bool tookDamage = players[i]->takeDamage(enemies[j]->getDamage()); //take damage and store if they did
            if (tookDamage) //if damage taken
            { //apply knockback
                //for knockback calculation
                AABB player = players[i]->getHitbox();
                AABB enemy = enemies[j]->getHitbox();
                //calcualate the direction away from the enemy
                float dirX = player.x - enemy.x;
                float dirY = player.y - enemy.y;
```

```
                float length = sqrt(dirX * dirX + dirY * dirY);
                if (length > 0.0f)
                {
                    dirX /= length; //normalise
                    dirY /= length;
                }

                //apply knockback
                players[i]->applyImpulse(dirX, dirY, enemies[j]->getKnockbackForce(),
    enemies[j]->getKnockbackDuration());
            }

        }
    }
```

Projectiles carry an `owner` pointer to the entity that fired them, so friendly projectiles can't hit other players and enemy projectiles can't hit enemies, so enemy projectiles are only tested against players.

```

        //if the owner of the projectile is the player
        if (projectile.owner == ProjectileOwner::Player)
        {
            //we want to check player projectiles check against all enemies
            for (int j = 0; j < enemies.getSize(); j++)
            {
                if (Collisions::checkAABBCollision(projectile.getAABB(), enemies[j]->
    getHitbox())) //projectile + enemy hitbox
                {
                    bool tookDamage = enemies[j]->takeDamage(projectile.damage); //enemy
    take damage from projectile
                    if (tookDamage) //if damage taken, apply knockback
                    {
                        enemies[j]->applyKnockback(projectile.dirX, projectile.dirY,
    projectile.knockbackForce, projectile.knockbackDuration);
                    }
                    projectile.dead = true; //mark projectile is dead
                    break; //hit an enemy so we should stop checking other enemies for this
     projectile
                }
            }

            //if it didn't hit an enemy, check spawners (no knockback, spawners don't move)
            if (!projectile.dead)
            {
                for (int j = 0; j < spawners.getSize(); j++)
                {
                    if (spawners[j]->isDead()) continue; //skip already destroyed spawners

                    if (Collisions::checkAABBCollision(projectile.getAABB(), spawners[j]->
    getHitbox())) //projectile + spawner hitbox
                    { //dont need to check for damage as spawners dont take knockback
                        spawners[j]->takeDamage(projectile.damage); //spawner takes damage
                        projectile.dead = true; //arrow is used up
                        break; //stop checking other spawners
                    }
                }
            }
        }
        else if (projectile.owner == ProjectileOwner::Enemy) //if the owner is an enemy (
    trap arrows count as enemy arrows too)
        //.... continued
```

This is fine at the entity counts Cryptmarch reaches, but the cost of doing so grows massively. A better alternative to this would have been to use a spatial grid or spatial hash. Where each entity is placed in the cell/tile it occupies, and only entities in the same or neighbouring cells are tested.

4.3 Tile collision & the flat tile array

Tiles are stored as a flat 1D array of tile IDs, indexed by index $= y \times \text{width} + x$. I went with the cache-friendly approach of using a 1D array instead of a 2D array of pointers. This way, the whole map sits in one contiguous block of memory, so drawing a row reads neighbouring memory. Each ID indexes a `TileDefinition` that holds the tile's sprite offset, whether it is walkable, and an optional base tile for drawing underneath transparent tiles. Later on in the project, I added a separate array of overrides for each tile which lets doors switch between blocked and walkable at runtime.

```
//          name  xpos(column)  ypos(row)  walkable  baseTile(optional)
//WALLS
/*  0*/ TileDefinition("DirtWallTop", 0, 0, false),
/*  1*/ TileDefinition("DirtWallSide", 1, 0, false),
/*  2*/ TileDefinition("RoughStoneWallTop", 0, 1, false),
/*  3*/ TileDefinition("RoughStoneWallSide", 1, 1, false),
/*  4*/ TileDefinition("StoneBrickWallTop", 0, 2, false),
/*  5*/ TileDefinition("StoneBrickWallSide1", 1, 2, false),
/*  6*/ TileDefinition("StoneBrickWallSide2", 2, 2, false),
// ... continued..
```

Converting a position to tile coordinates is $O(1)$ by dividing by the tile size. `isBlockedByWall` converts a box's edges to tile coordinates and checks only the tiles it overlaps.

```
bool Map::isBlockedByWall(float posX, float posY, int width, int height) const
{
    //find the edges of the object in world space
    //-1 since 32 wide box from x=0 is 0-31, pixel 32 is the next tile.
    float left = posX; //left edge
    float right = posX + width - 1.0f; //right edge
    float top = posY; //top edge
    float bottom = posY + height - 1.0f; //bottom edge

    //convert pixel coordinates of edges into tile grid coordinates
    //EG:   4
    //    3   5
    //      5
    //using floor division now for negative coordinates
    int minTileX = Math::floorDiv(static_cast<int>(floor(left)), tilePixelSize);
    int maxTileX = Math::floorDiv(static_cast<int>(floor(right)), tilePixelSize);
    int minTileY = Math::floorDiv(static_cast<int>(floor(top)), tilePixelSize);
    int maxTileY = Math::floorDiv(static_cast<int>(floor(bottom)), tilePixelSize);

    //loop through all the tiles that the object overlaps
    for (int tileY = minTileY; tileY <= maxTileY; tileY++)
    {
        for (int tileX = minTileX; tileX <= maxTileX; tileX++)
        {
            //get the tile at current position
            int id = getTileAt(tileX, tileY);
            if (id == -1 || id >= tileCount) return true; //check for invalid tile, will
    treat as out of bounds for now so we want it to act like an invisible wall

            if (!infinite) //if not infinite
            { //blockedTiles is nullptr on infinite mode, so we only check on non-infinite
    mode
                //get the index for blocked tiles
                int index = tileY * tilemapWidth + tileX;
```

```
                if (blockedTiles[index]) return true; //if that tile is blocked return true
            }

            //check if this tile is not walkable
            if (!tileDefinitions[id].walkable) return true;
        }
    }
    return false; //else its walkable
}
```

An alternative, which was my first approach, looped over every wall tile, which was $O(w)$ per check for w walls and would have meant thousands of checks per frame.

4.4 Data-driven level loading

Levels are loaded from text files. The first line holds the width and height, followed by the tile data, which is represented as width × height tile IDs, then tags that place objects, such as `SPAWNER x y`, `DOOR x y id` or `KEY x y`. Each value is read with », so loading is $O(w \times h)$, and unknown tags are logged as errors to catch typos.

```
//read and extract width and height from the first line
mapFile >> width >> height;

//update width/height
tilemapWidth = width;
tilemapHeight = height;
//get the world size in pixels by multiplying the tile size by the tilemap
mapWidth = width * tilePixelSize;
mapHeight = height * tilePixelSize;
//allocate array of new map
tilemapData = new int[tilemapWidth * tilemapHeight];
//allocate the blocked array, the () at the end sets every value to false
blockedTiles = new bool[tilemapWidth * tilemapHeight]();

//read the data from the file and insert it into the array
for (int i = 0; i < tilemapWidth * tilemapHeight; i++)
{
    mapFile >> tilemapData[i];

    //protect against typos in the map file, an unknown ID would read outside
    tileDefinitions and crash
    if (tilemapData[i] < 0 || tilemapData[i] >= tileCount)
    {
        Logger::Error("Invalid tile ID " + std::to_string(tilemapData[i]) + " at slot " +
    std::to_string(i) + " in " + path);
        tilemapData[i] = 13; //replace with GreyFloorBlank so the game can still run
    }
}

//entities spawned via tilemap
std::string tag;
while (mapFile >> tag) //>> will return false when nothing is left, so the loop stops
{
    if (tag == "SPAWNER")
    {
        int tileX, tileY;
        mapFile >> tileX >> tileY;
        spawnerPositions.add({ (float)(tileX * tilePixelSize), (float)(tileY *
    tilePixelSize) });
    }
    else if (tag == "PLAYER")
    {
        int tileX, tileY;
```

```
41          mapFile >> tileX >> tileY;
42          playerStartPositions.add({ (float)(tileX * tilePixelSize), (float)(tileY *
        tilePixelSize) });
43      }
44      //... etc for other tags
45      else
46      {
47          Logger::Error("Unknown tag in map file: " + tag); //catches typos or invalid tags
48      }
```

An alternative would have been to use a binary format, which is faster to read and smaller on a disk. I went with text files because they are easier to work with. Unfortunately, this approach is not very human-friendly, and I had to generate the levels using AI.

4.5 Fixed vs infinite world

The fixed map has bounds checks, and anything outside the grid is treated as an invisible wall. The infinite world is built from eight AI generated 16×16 chunk templates. We can find out what chunk we are in with floor division, and where we are inside the chunk with positive modulo. Both are needed because coordinates can be negative. Each template is randomly generated by a spatial hash of the chunk coordinates [4]:

```
1 if (chunkX == 0 && chunkY == 0) return 0; //force a 'safe' starting room with no spawners
2 unsigned int hashVal = (ux * 73856093U) ^ (uy * 19349663U) ^ us; //ux, uy, us = chunk x,
      chunk y, seed
3 return hashVal % templateCount;
```

A hash is a pre-determined function, so the same chunk coordinates always give the same template. Walking away and coming back regenerates exactly the same layout without storing any chunks, and memory use stays constant however far the players travel. An alternative, which is what I started with, was a single, bigger map repeated with modulo. It was much simpler to implement, but it got very repetitive, and I wanted to make the infinite world more interesting.

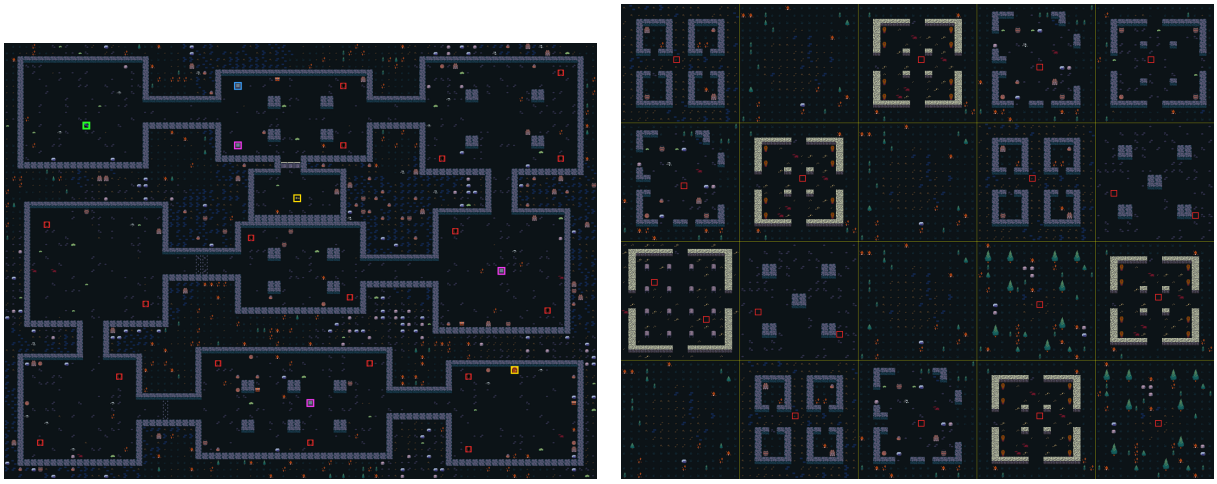


Figure 7: Left: a fixed level loaded from file. Right: hashed chunk templates stitched into the infinite world

5 Collectables, Traps & Advanced Features

5.1 Potions, keys, doors and pressure plates

```
1 enum class PickupType { Health, Mana, Key };
2
3 class Pickup : public Entity
```

```
{
private:
    PickupType type; //health or mana
    int amount; //how much the potion restores

    float lifetime = 20.0f; //seconds before it disappears

    bool collected = false; //set when the player picks it up
    bool expires; //potions disappear after their lifetime, the key never does

public:
    //_expires has a default value, so the existing potion code doesnt need to change
    Pickup(EntityStats _stats, GamesEngineeringBase::Image* _image, PickupType _type, int
    _amount, bool _expires = true) : Entity(_stats, _image)
    {
        type = _type;
        amount = _amount;
        expires = _expires;
    }

    PickupType getType() const { return type; }
    int getAmount() const { return amount; }

    void collect() { collected = true; }

    //ready to be deleted if it was picked up or ran out of time
    bool shouldRemove() const { return collected || (expires && lifetime <= 0.0f); }

    void update(GamesEngineeringBase::Window& window, Map& map, float deltaTime) override
    {
        lifetime -= deltaTime; //count down
    }

    void draw(GamesEngineeringBase::Window& window, Camera& cam) override
    {
        drawSprite(window, cam);
    }
};
```

Enemies have a 30% chance to drop a potion when they die, with a 50% chance to be either a health or mana potion. Each level in story mode has a key. Picking it up opens the exit door, and walking through the open exit door completes the level.

Pressure plates and doors are linked by an ID in the map file. When a plate is pressed, `openDoorsWithId` loops over the doors and opens every door with a matching ID.

```
//opens doors with corresponding id
void EntityManager::openDoorsWithId(int id, Map& map)
{
    for (int i = 0; i < doors.getSize(); i++) //get all doors
    {
        //if the door isnt an exit door and the door has the right id
        if (!doors[i]->getIsExit() && doors[i]->getId() == id) //this is because we want
    the key to open the final door, shouldnt happen anyways but i included it for safety
        {
            doors[i]->open(map); //open the door
        }
    }
}
```

This is $O(k)$ for k doors, which is fine with a handful per level, but the alternative of a lookup table indexed by ID would make it $O(1)$. Opening a door swaps its sprite and clears its blocked-tile

override, so players, enemies and projectiles can pass through.

5.2 Traps

There are two forms of traps: spike traps and fake pressure plates. Spike traps cycle between up for 1 second and down for 1.5 seconds, and hurt a player standing on them while they are up, at most once per cycle.

```
class SpikeTrap : public Entity
{
private:
    bool spikesUp = false; //if the spikes are up or not
    float timer; //time left in the current up/down phase
    const float upTime = 1.0f; //seconds the spikes stay up
    const float downTime = 1.5f; //seconds the spikes stay down (safe to cross)
    int damage = 20; //damage the spikes deal
    bool hasHit = false; //only hurt once each time the spikes come up, otherwise standing
    on it would drain health very quickly

    //sprite positions
    const int downTileX = 15;
    const int upTileX = 16;
    const int trapTileY = 16;

public:
    SpikeTrap(EntityStats _stats, GamesEngineeringBase::Image* _image);

    bool canHit() const { return spikesUp && !hasHit; } //dangerous right now
    void markHit() { hasHit = true; } //sets has hit to true
    int getDamage() const { return damage; } //gets the damage of the spikes

    void update(GamesEngineeringBase::Window& window, Map& map, float deltaTime) override;
    void draw(GamesEngineeringBase::Window& window, Camera& cam) override;
};
```

I wanted a separate type of trap, that worked more like a trigger, so I added fake pressure plates. They look identical to real ones but fire arrows when stepped on. They inherit from the same class as real pressure plates, since the base behaviour of being pressed is the same, but contain an isFake flag to differentiate them, which changes the behaviour of them in the `EntityManager`.

```
void EntityManager::updateLevelObjects(GamesEngineeringBase::Window& window, Map& map,
    float deltaTime)
{
    //PRESSURE PLATES
    for (int i = 0; i < plates.getSize(); i++) //for each plate
    {
        PressurePlate* plate = plates[i]; //get the plate
        plate->update(window, map, deltaTime); //update the fake plates for reset

        for (int j = 0; j < players.getSize(); j++) //get players
        {
            if (!players[j]->checkEntityCollision(plate)) continue; //check if being stood
    on

            if (plate->press()) //true only on the frame the plate goes down
            {
                if (plate->getIsFake()) //if its a fake
                {
                    Logger::Debug("Fake pressure plate triggered, firing arrows");
                    fireArrowTrap(plate->getCentrePoint(), map); //fire arrows
                }
                else
                {
```

```
22                      Logger::Debug("Pressure plate " + std::to_string(plate->getId()) + "
    pressed");
23                      openDoorsWithId(plate->getId(), map); //open doors related to id
24                  }
25              }
26          }
27      }
28      //...
```

Both traps originally triggered every frame, which I fixed with a once-per-cycle hit flag and a reset timer.

5.3 Level progression

The `GameController` holds a list of level files.

```
1  //levels are played in this order
2  static const int levelCount = 2; //amount of levels we have
3  inline static const std::string levelPaths[levelCount] = //string defs for levels
4  {
5      "resources/maps/map_dungeon.txt",
6      "resources/maps/map_catacombs.txt",
7  };
8  int currentLevel = 0; //what level we are currently on
```

When a level ends, the tally screen shows whether the level was completed, if all the players died, or if the game was finished. The tally screen also shows kills and time survived, and the level number. Then the next level loads, if there is a next level. Kills, time and health carry over between levels in the story mode, which makes later levels harder due to the time-related spawning.

5.4 Multiplayer

In multiplayer, each player has an ID, their own controls, and their own health and mana bars on opposite sides of the screen. Enemies target the nearest player in range, which is $O(p)$ per enemy.



Figure 8: Multiplayer with two players, each with their own health and mana bars

A leash on the camera keeps both players on one screen by preventing movement offscreen, the camera follows both players, using a midpoint between them both as part of its `follow` function. Players are looked up by ID for a specific player (for example the HUD) and by index for “every living player”, because `removeSwap` reorders the list when a player dies.

5.5 Saving and loading

Saving is not implemented. My plan was to use serialisation, by converting the game's current state into values which I would then write to a file. The save file would contain a header (mode, level index, and the seed for infinite mode), then each player (ID, hero, position, health, mana), then each entity list as a count followed by one line per entity (type, position, health, and state such as a door being open or a plate pressed). Saving and loading would both be $O(\text{number of entities})$. To load, the game would load the map, clear the `EntityManager`, and recreate each entity from its line. I wanted to implement a pause screen and a load, save menu, but I ran out of time.

6 Evaluation

All measurements were taken with the in-game FPS counter.

Build stage	Release FPS	Release ms	Debug FPS
First tile renderer (32×24 map)	600	1.67	60–70
Unchecked pixel access	925	1.08	130
Pointer walking blending, duplicate writes removed	1800	0.56	700
New 48×36 map with tile collision	1800	0.56	700
New maps with base tiles	1600	0.63	500
Infinite map	1600	0.63	500
Final version	1600	0.63	500

Table 1: Frame rate at each stage of development (measured without entity collisions)

Optimising the renderer roughly tripled the Release frame rate and gave a tenfold improvement in Debug. Moving to a larger map with tile collision made almost no change, which matches tile collision being $O(1)$ per check and drawing being $O(\text{visible tiles})$ rather than $O(\text{map size})$. The new maps cost about 200 FPS, which I believe is due to base tiles being used, where each transparent tile draws its base tile underneath it first, so areas with lots of scenery make roughly twice as many draw calls. The infinite map performed the same as the fixed map, which matches the chunk hash being $O(1)$ per tile lookup.

	Release		Debug	
Enemies	FPS	ms per frame	FPS	ms per frame
1	1565	0.639	481	2.080
50	1498	0.668	475	2.106
100	1457	0.687	478	2.093
200	1385	0.722	447	2.236
500	1238	0.808	377	2.653

Table 2: Average frame time against enemy count

Stress testing was done by adding a debug key that spawns enemies, and recording the average frame time. The growth is roughly linear, which matches $O(p \cdot e)$ with one player and no projectiles, effectively $O(e)$. Using attacks made the FPS drop by around 50-100FPS, which increased as there were more enemies, which is expected because attacks spawn projectiles, which are checked against every enemy (Section 4.2), which would be better with a spatial grid (Section 8).

7 Limitations

- Saving and loading were planned but not built.
- Enemies chase in a straight line, so they get stuck on walls between them and the player.
- Entity-vs-entity collision is brute force, so the cost grows with projectiles $\times$ enemies.
- Some entities carry data they don't use, such as health on traps, keys have unused amount values.
- Map dimensions are typed by hand into the file header, so a mistake there breaks the level.

8 What I Would Do Differently

I would build a spatial grid from the start, so that collision and targeting only check nearby entities, saving me a lot of performance. I would also design the save system early by giving `Entity` a virtual `serialise` function, so every new entity type would be saveable as it was written rather than retrofitted at the end. Finally, I would record frame times at fixed points throughout development, including a stress test, so the evaluation could test my Big-O claims directly.

9 Conclusion

Cryptmarch is a *Gauntlet*-style game with three heroes, six enemy types, with traps, keys and doors, and both fixed and infinite worlds through a story mode and infinite mode. The main data structure is my own `Vector<T>` dynamic array, used for every entity list. The map uses a flat tile array that gives $O(1)$ tile lookup and collision. Key algorithmic choices were AABB collision with test-then-move, weighted random spawning, a bounded insertion leaderboard for top-N targeting, and a spatial hash that makes the infinite world deterministic without storage. The biggest improvements left are a spatial grid for collision and a working save system.

References

[1] Atari Games (1985) *Gauntlet* [Arcade game]. Atari Games.

[2] Seth (2024) *32rogues* [Sprite and tile set]. Available at: https://sethbb.itch.io/32rogues.

[3] Kenney (n.d.) *Desert Shooter Pack* [Game assets, CC0]. Available at: https://kenney.nl/assets/desert-shooter-pack.

[4] Teschner, M., Heidelberger, B., Müller, M., Pomeranets, D. and Gross, M. (2003) 'Optimized spatial hashing for collision detection of deformable objects', *Proceedings of Vision, Modeling, and Visualization (VMV 2003)*, pp. 47–54.